GOVERNMENT POLYTECHNIC PUNE

(An Autonomous Institute of Government of Maharashtra)
University Road, Shivajinagar, Pune-411016

DEPARTMENT OF COMPUTER ENGINEERING

Academic Year 2026-27

A Micro Project Report On

"Intelligent Airport Flight, Runway & Priority Management Simulator"

Based on Course
Object Oriented Programming

Submitted By
Kamble Vaibhav Balasaheb (2506084)

Guided by:
Usha C Khake Ma'am
Lecturer, Government Polytechnic Pune

DEPARTMENT OF COMPUTER ENGINEERING
Academic Year 2026-27

Certificate

This is to certify that Mr/Mrs. KAMBLE VAIBHAV BALASAHEB, Enrollment No. 2506084, a student of the Third Semester Diploma in Computer Engineering at Government Polytechnic, Pune, has satisfactorily completed the Micro Project in the course Object Oriented Programming (CM31203) for the academic year 2026-2027, as prescribed in the curriculum.

_____

Signature of Project Guide

Usha C Khake Ma'am
Lecturer, Government Polytechnic Pune

Index

Sr no	Topics	Page
1.	ACKNOWLEDGEMENT	04
2.	RATIONALE	05
3.	AIM AND OBJECTIVES	06
4.	THEORY	07
5.	METHODOLOGY	08
6.	REFERENCES & COs	09
7.	CONCLUSION	10

Group Details

Sr no	Name	Enrollment no	Remark
1	Kachare Shrutika Sunil	2506078	
2	Kamble Ayush Ramesh	2506082	
3	Kamble Vaibhav Balasaheb	2506084	
4	Karale Anushka Satish	2506085	

Acknowledgement

I extend my deepest gratitude to Dr. Rajendra Patil, our esteemed Principal, whose unwavering support, and encouragement propelled me through the journey of my project, “Intelligent Airport Flight, Runway & Priority Management Simulator”. His vision and guidance paved the path for its successful completion.

Special recognition is due to Mrs. J. R. Hange, our dedicated Head of Department, for her invaluable insights and constant motivation, which played a pivotal role in shaping this endeavour. A profound thank-you to Usha C Khake Ma'am, our mentor, whose boundless patience, and unwavering support were the cornerstones of my journey. Your invaluable advice and keen insights were instrumental in overcoming obstacles and achieving milestones. Your guidance will forever resonate with me.

Furthermore, I acknowledge that the completion of this project was a testament to my dedication and perseverance. While I received support and guidance from esteemed individuals, the execution and culmination of this endeavor were solely my own.

In closing, I express my deepest gratitude to all those who contributed to this endeavor, directly or indirectly. This project stands as a testament to collaboration, determination

Rationale

In busy modern airports, manually tracking flight schedules, assigning runways, and managing emergency arrivals leads to operational delays, increased human error, and heightened safety risks. Conventional procedural approaches often lack the modularity required to handle complex real-time decision-making, such as priority queues for low-fuel aircraft.

The Airport Runway & Flight Manager system addresses these issues by centralizing flight monitoring and runway allocation into a single C++ application. By utilizing Object-Oriented Programming (OOP), the project models real-world entities—such as flights, runways, and priority queues—into software objects, providing structured data management, automated emergency detection, and efficient runway scheduling.

Furthermore, air traffic controllers require immediate visibility into critical parameters like aircraft fuel levels and runway maintenance statuses. By replacing fragmented, manual logs with an integrated computational system, operators can automatically re-evaluate queues whenever an incoming flight experiences low fuel reserves, ensuring rapid response times during high-stress scenarios.

Ultimately, this software provides a robust foundation for scalable airport resource management. Implementing data structures and class-based abstractions allows new capabilities, such as multi-terminal support or advanced conflict detection, to be seamlessly integrated into the software architecture without disrupting existing core operational logic.

"Intelligent Airport Flight, Runway & Priority Management Simulator"

Aim :

To design and implement a centralized Airport Runway & Flight Manager system in C++ using object-oriented programming concepts to streamline flight tracking, runway allocation, and emergency priority queuing.

Objectives :

Develop an object-oriented C++ application to centralize flight and runway data management.

Implement automated priority queue processing based on flight priority and fuel emergency levels.

Manage runway states dynamically during flight landing and takeoff operations.

Ensure persistent storage of system configurations and logs using C++ file streams.

Generate comprehensive airport reports detailing flight histories, fuel emergencies, and runway utilization statistics.

Tools & Technologies Used :

C++ Language
Object-Oriented Programming (OOP)
Standard Template Library (STL)
File Handling (fstream)
Console UI / CLI

Theory

Object-Oriented Programming (OOP) is a paradigm centered around objects containing data fields and associated methods. Key OOP concepts applied in this system include Encapsulation, abstraction, inheritance, polymorphism, and file handling.

Classes serve as blueprints for objects such as flights and runways, bundling attributes and behavioral functions together. Inheritance enables class hierarchies to share common characteristics, while polymorphism allows functions like priority calculation or display formatting to behave dynamically based on object types. File stream handling (fstream) ensures data persistence across sessions by reading and writing records to binary dat files.

Encapsulation safeguards internal variables—such as flight IDs, aircraft types, and runway occupancy statuses—by restricting direct modification and providing access exclusively through defined member functions. Constructive use of destructors and constructors facilitates proper memory management during object instantiation and cleanup.

Additionally, runtime polymorphism via virtual functions allows the system to process general flight and runway entities through uniform interface calls, adapting dynamically depending on specific subtypes. The C++ Standard Template Library (STL) stream utilities streamline formatted input/output operations and record-keeping, forming the underlying infrastructure for reliable disk persistent storage.

Methodology

The development of the Airport Runway & Flight Manager follows a modular object-oriented workflow in C++.

1. Requirement & Structure Design: System requirements were defined around core objects—Flights, Runways, Schedules, and Priority Queues. Classes were structured with private member variables and public member functions for controlled access.

2. Flight & Fuel Monitoring Implementation: The system records details including Flight ID, Airline, Fuel Level, and Route. An automated condition evaluates fuel percentages; if fuel falls below the emergency threshold (e.g., 20%), the system automatically elevates the flight priority to EMERGENCY.

3. Priority Queue & Runway Allocation: Flights are pushed into a dynamic priority queue sorted by emergency status, priority levels, and submission order. Available runways (categorized as Landing, Takeoff, or Mixed) are assigned based on suitability and status (AVAILABLE, OCCUPIED, MAINTENANCE).

4. File Processing & Report Generation: Data persistence is maintained by reading and writing binary data files (flights.dat, runways.dat, schedule.dat, airport.dat). Comprehensive reporting modules compile runway utilization metrics, emergency logs, and daily flight counts.

References

E. Balagurusamy, Object Oriented Programming with C++, 8th ed., McGraw Hill Education, 2020.

Bjarne Stroustrup, The C++ Programming Language, 4th ed., Addison-Wesley, 2013.

C++ Reference Documentation - Standard C++ Library Filesystem and I/O Streams, cppreference.com

Course Outcomes

COs	Outcomes
1	Understand procedural and object-oriented paradigms.
2	Implement different functions in OOP.
3	Develop programs using classes and objects.
4	Implement programs on inheritance.
5	Apply concepts of polymorphism and type conversion.
6	Develop applications for file handling.

Conclusion

The Airport Runway & Flight Manager successfully demonstrates the application of object-oriented programming concepts to solve complex real-world scheduling problems. By encapsulating flight details, runway states, and operational statistics within C++ classes, the project provides an efficient, modular solution for airport centralization.

The implemented priority queue ensures safety by automatically prioritizing emergency landings for low-fuel aircraft. Persistent data handling via standard C++ file streams guarantees that flight logs, schedule records, and airport settings are saved across sessions. Overall, the project met all core software design objectives and reinforced concepts of object-oriented design, file processing, and menu-driven system architecture.